01 파이썬 기반의 머신러닝과 생태계 이해

01. 머신러닝의 개념

머신러닝이란?

애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법.

머신러닝 알고리즘은 데이터를 기반으로 예측 오류를 최소화 하여 패턴을 인지하고 결과를 도출함.

→ 데이터가 중요할까 알고리즘이 중요할까?

머신러닝의 분류

- 지도학습 : 분류, 회귀 등
- 비지도학습
- 강화학습

02. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

사이킷런 (Scikit-Learn), 넘파이, 판다스, 맷플롯립, 시본

03. 넘파이

선형대수 기반의 프로그램을 쉽게 만들 수 있도록 하는 패키지,

장점:

- 빠른 배열 연산 속도 (루프 사용하지 X)
- C/C++ 등 호환 API 제공 → 데이터 통합 가능

수행 성능이 중요한 부분은 C/C++로 작성하고 넘파이에서 호출하면 됨.

많은 머신러닝 알고리즘이 넘파이 기반으로 작성됐으며, 알고리즘의 입력 데이터와 출력 데이터를 넘파이 배열 타입으로 사용한다.

넘파이의 기반 데이터 타입은 ndarray, 이를 통해 다차원 배열을 쉽게 생성

- array() 함수는 입력된 인자 → ndarray로 변환
- ndarray배열의 shape 변수를 통해 배열 차원 알 수 있음

ndarray의 데이터 타입

숫자형 : int형, unsigned int형, float형

같은 데이터 타입 내에서 연산 가능, 한 개의 ndarray 객체에는 int와 float 공존 X

정수형과 문자형이 함께 있을 경우 정수형을 문자형 취급

astype()을 이용해서 데이터 타입을 바꿈 (메모리를 절약하기 위해)

ndarray를 편리하게 생성하기 - arange, zeros, ones // reshape()

arange() : array를 range로 표현한 것

zeros() : 튜플 형태의 shape 값 입력 → 모든 값을 0으로 채운 해당 shape의 ndarray 반환

ones() : 튜플 형태의 shape 값 입력 → 모든 값을 1으로 채운 해당 shape의 ndarray 반환

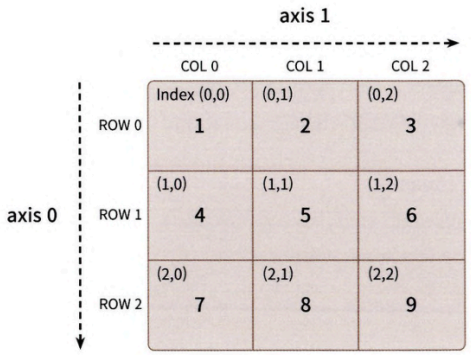
!! dtype 지정 안 할 경우 default float64 형

reshape() : ndarray를 특정 차원 및 크기로 변환

ㄴ 차원이 맞지 않을 경우 error, -1을 이용하기도

인덱싱

- **특정한 데이터만 추출**: 원하는 위치의 인덱스 값 반환



row, col이 아닌 axis

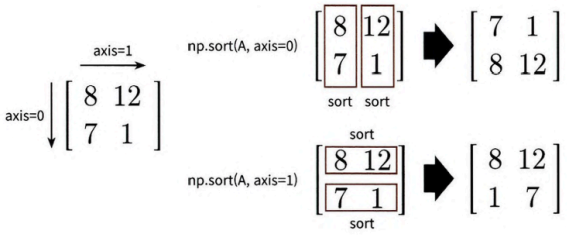
- **슬라이싱(Slicing)**: 연속된 인덱스상을 추출하는 방식. ':'를 사용하여 범위 추출 가능 // 2 차원 이상을 슬라이싱 할 경우 공통된 row, col만 추출됨.
- **팬시 인덱싱(Fancy Indexing)**: 인덱싱 집합을 리스트 혹은 ndarray 형태로 지정해 반환.
- **불린 인덱싱(Boolean Indexing)**: 특정 조건 혹은 True/False 통해 반환.

행렬의 정렬 - sort()와 arsort()

np.sort() : 원 행렬은 그대로, 이번에만 정렬

ndarray.sort() : 원 행렬 자체를 변환, 반환 값 None.

행렬이 2차원 이상일 경우에 axis 축 값 설정을 통해 로우 방향, 또는 칼럼 방향으로 정렬을 수행할 수 있습니다.



정렬된 행렬의 인덱스를 반환하기

np.arsort() : 원소 값이 아닌 원본 원소의 인덱스를 변환.

팬시 + 불린 인덱스를 함께 자주 사용함.

선형대수 연산

행렬의 연산 dot, 전치 행렬 등을 numpy의 dot(), transpose()를 이용해 구할 수 있음.

04. 데이터 핸들링 - 판다스

2차원 데이터를 효율적으로 처리할 수 있는 훌륭한 기능 제공.

판다스의 시작

pd.read_csv()를 이용해 파일을 DataFrame 객체로 변환.

info(), describe()를 통해 데이터의 특성을 대략적으로 알 수 있음.

value_counts() 호출시 칼럼값의 유형과 건수 확인 가능; 많은 순서대로 정렬되어 반환, null은 무시 (dropna로 판)

Series()를 이용해 한 개 칼럼만 뽑아내기 가능

!! 모든 인덱스는 고유성 보장.

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환, 수정

pd.DataFrame(), df_dict.values.tolist(), df_dict.to_dict() 등

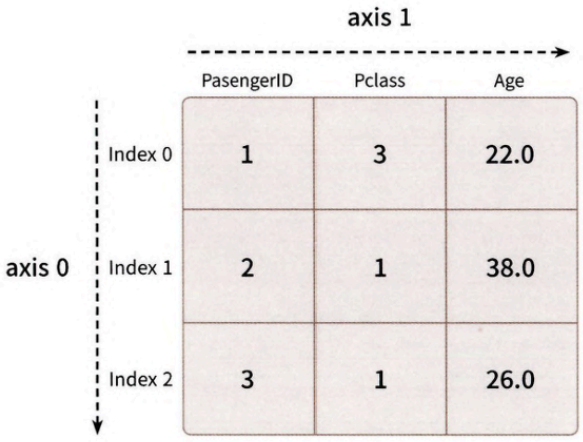
DataFrame과의 상호 변환이 자유롭게 이루어짐

[]연산자를 통해 수정 및 삭제 가능

DateFrame 데이터 삭제

기본 설정)

DataFrame.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise')



〈 DataFrame에서의 axis 구분 〉

axis = 0은 로우 방향, axis = 1은 칼럼 방향

inplace 설정을 통해 원소 삭제/칼럼 전체 삭제 설정

Index 객체

단일 값 반환 및 슬라이싱 가능, but 객체는 함부로 변경할 수 없음.

연산에서 제외되며 오직 식별용

데이터 셀렉션 및 필터링

(넘파이의 단일 값 추출, 슬라이싱, 팬시 인덱싱 등의 역할)

*데이터프라임에서 []엔 칼럼명 문자 혹은 인덱스 변환 표현식만 가능

iloc[] : 열 위치 (ex. data_df.iloc[:,3])

loc[] : 명칭 기반 데이터 추출 (ex. data_df.loc['one1, 'Name'])

iloc[], loc[]의 특징과 주의점

- 여러 칼럼 값 추출시 dataframe[’칼럼명’]으로 충분
- 인덱스 혹은 칼럼명 - 명칭 기반 인덱싱
  열의 위치 등 - 위치 기반 인덱싱

불린 인덱싱을 통해서 추출 가능

수식 설정

- **and** 조건일 때는 **&**
- **or** 조건일 때는 **|**
- **Not** 조건일 때는 **~**

*

titanic_df[(titanic_df ['Age']> 60) & (titanic_df['Pclass']==1) & (titanic_df['Sex']=='female')]

↔

```
cond1 = titanic_df ['Age']> 60
cond2 = titanic_df['Pclass']==1
cond3 = titanic_df['Sex']=='female'
titanic_df[ cond1 & cond2 & cond3]
```

ㄴ 본인 선택

정렬 적용

ascending = True : (기본 설정) 오름차순

ascending = False : 내림차순

Aggregation 함수 적용

특별한 설정 없을 경우 전체 결과 반환

대상 칼럼 추출 후 적용

groupby() 적용

groupby 적용시 대상 칼럼을 제외한 모든 칼럼에 aggregation 함수 적용

ㄴ 대상 칼럼 순서로 그룹화 되어 정렬 → 나머지 칼럼 함수 계산 되어짐

필요한 칼럼만 필터링 후 count 가능

(ex. titanic_df,groupby('Pclass')[['PassengerId', 'Survived']].count())

결손 데이터 처리하기

1. isna()로 결손 데이터 여부 확인
2. fillna()로 결손 데이터 대체하기

apply lambda 식으로 데이터 가공

함수를 이용하지 않고 간결하게 식으로 정리

lambda x ： x ** 2

받을 인자를 선언과 수식 한번에 설정

길어질 경우 map(lambda x ： x ** 2) 결합 가능

lambda는 if else 지원

*반환값을 먼저 적기

(ex. lambda x ： if x〈=15 'Child' else 'Adult’ (X)

lambda x : 'Child' if x〈=15 else 'Adult' (O))

*else if는 지원X, 첫 if else 괄호 안에서 또 if else를 만들어야 함 → 조건이 길어질 경우 함수를 만드는 게 더 편리함.

정리

넘파이와 판다스 매우 중요합니다.